

👻 SPECTRE: Hybrid Ordinary-Parallel Speculative Serving for Resource-Efficient LLM Inference

¹Jincheng Xie ²Yawen Ling ²Qi Xiao
²Feiyu Zhang ¹Zhongyi Huang ²Wen Hu* ³Yu Zheng*

¹Tsinghua University ²AI Infra Team at JDT
³JD iCity, JD Technology, JD Intelligent Cities Research
 xiejc22@mails.tsinghua.edu.cn {lingyawen1, xiaoqi.31, zhangfeiyu.17}@jd.com
 zhongyih@tsinghua.edu.cn msyuzheng@outlook.com huwen.31@jd.com
 *Corresponding Author

Abstract

LLM serving platforms are increasingly deployed as multi-model cloud systems, where user demand is often long-tailed: a few popular large models receive most requests, while many smaller tail models remain underutilized. We propose **SPECTRE** (Parallel **SP**eculative Decoding with a Multi-Tenant **RE**mote Drafter), a serving framework that reuses underutilized tail-model services as remote drafters for heavily loaded large-model services through speculative decoding. SPECTRE enables draft generation and target-side verification to run in parallel, and makes such parallelism effective through three techniques: a hybrid ordinary-parallel speculative decoding strategy guided by a threshold derived from throughput analysis, speculative priority scheduling to preserve draft-target overlap under multi-tenant traffic, and draft-side prompt compression to reduce draft latency. We implement SPECTRE in **SGLang** and evaluate it across multiple draft-target model pairs, reasoning benchmarks, real-world long-context workloads, and a wide range of batch sizes. Results show that SPECTRE consistently improves large-model serving throughput while causing only minor interference to the native workloads of tail-model services. In large-model deployments, including Qwen3-235B-A22B with TP=8, SPECTRE achieves up to **2.28× speedup** over autoregressive decoding and up to an additional **66% relative improvement** over the strongest speculative decoding baselines. Talk is cheap, we show you the code: <https://github.com/sgl-project/sglang/pull/22272>.

Date: May 13, 2026

1 Introduction

Large language model (LLM) serving platforms[1, 2] are increasingly deployed as multi-model cloud systems, where shared infrastructure supports models with different sizes, capabilities, and service roles [3]. In practice, user demand in such systems is often long-tailed: a small number of popular large models receive most requests, while many smaller models in the tail see much lighter traffic [4]. As these tail models remain online to serve the full model portfolio, their own traffic often falls short of fully utilizing their generation capacity. This imbalance motivates the reuse of idle tail-model capacity to assist heavily loaded large-model services.

In this paper, we investigate this opportunity through speculative decoding, a lossless acceleration method.

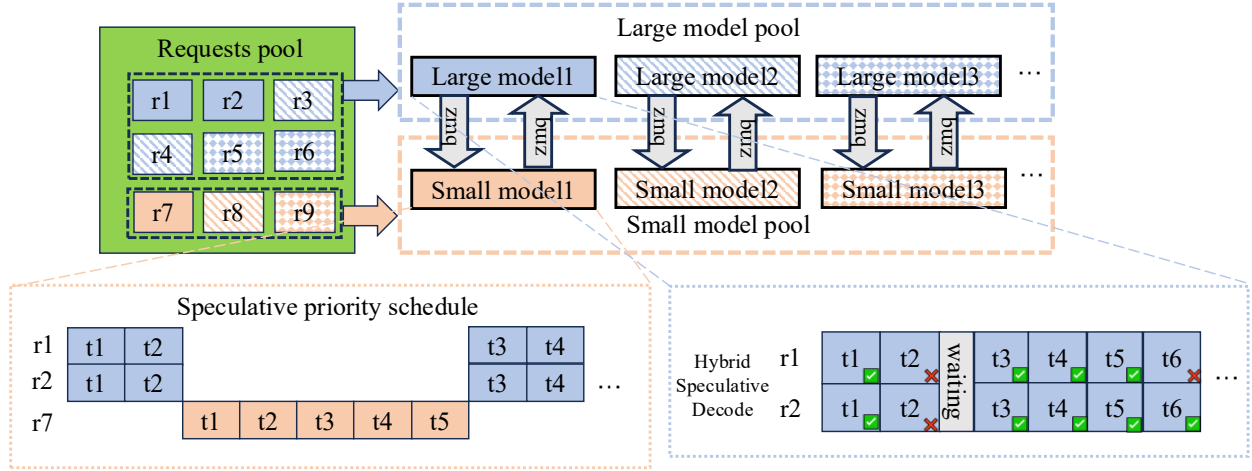


Figure 1 A large-model service obtains speculative drafts from an underutilized tail-model service via ZMQ. Blue requests correspond to large-model (target) queries, while orange requests correspond to small-model (draft-side normal traffic) queries. Each request is routed to the service instance with the matching pattern (hatched style). The draft-side system continues serving normal traffic while handling target-issued draft requests under speculative priority scheduling. SPECTRE further applies hybrid ordinary-parallel speculative decoding, dynamically switching modes based on a threshold derived from throughput analysis.

Crucially, this setting also creates an opportunity to parallelize draft generation and target-side verification, allowing the two stages to overlap [5, 6] instead of being serialized. However, such parallelism is not always beneficial, as it may reduce the acceptance length in speculative decoding, thereby diminishing the effective speedup and making its coordination a central systems challenge.

To address these challenges, we present **SPECTRE** (Parallel **SP**eculative **DE**coding with a Multi-**T**enant **RE**mote **D**rafter), a framework shown in Figure 1 that reuses underutilized tail-model services as remote drafters for heavily loaded large-model services while allowing these services to continue serving their native workloads. SPECTRE makes such parallelism effective through three techniques. First, it adopts a hybrid ordinary-parallel speculative decoding strategy that switches between the two coordination modes based on a threshold derived from our throughput analysis, using parallel execution only when it remains beneficial for the current batch (§ 2). Second, because speculative drafts are produced by a shared draft-side system, SPECTRE prioritizes speculative draft requests to preserve effective overlap between drafting and verification under multi-tenant traffic. Third, when target-side verification outpaces draft generation, SPECTRE compresses the draft-side prompt to reduce draft latency (§ 3.2). Together, these techniques make such parallelism effective in long-tail multi-model serving.

Extensive experiments across multiple draft-target model pairs, reasoning benchmarks, real-world long-context workloads, and a wide range of batch sizes show that SPECTRE consistently improves throughput. We further find that moderate draft-side load causes little degradation to target throughput, supporting the practicality of reusing shared tail-model services as remote drafters. For large-model deployments, including Qwen3-235B-A22B with TP=8, SPECTRE achieves up to **2.28× speedup** over autoregressive decoding and up to an additional **66% relative improvement** over the strongest speculative decoding baselines. At the system level, these throughput gains translate into up to **81% higher revenue** than autoregressive decoding.

In summary, this paper makes the following contributions:

- We propose **SPECTRE**, a serving framework that reuses underutilized tail-model services as remote drafters for heavily loaded large-model services while allowing these services to continue serving their native workloads.
- We design a hybrid ordinary-parallel speculative decoding strategy for SPECTRE, whose switching rule is guided by a threshold derived from our throughput analysis so that parallel coordination is used only when it remains beneficial.

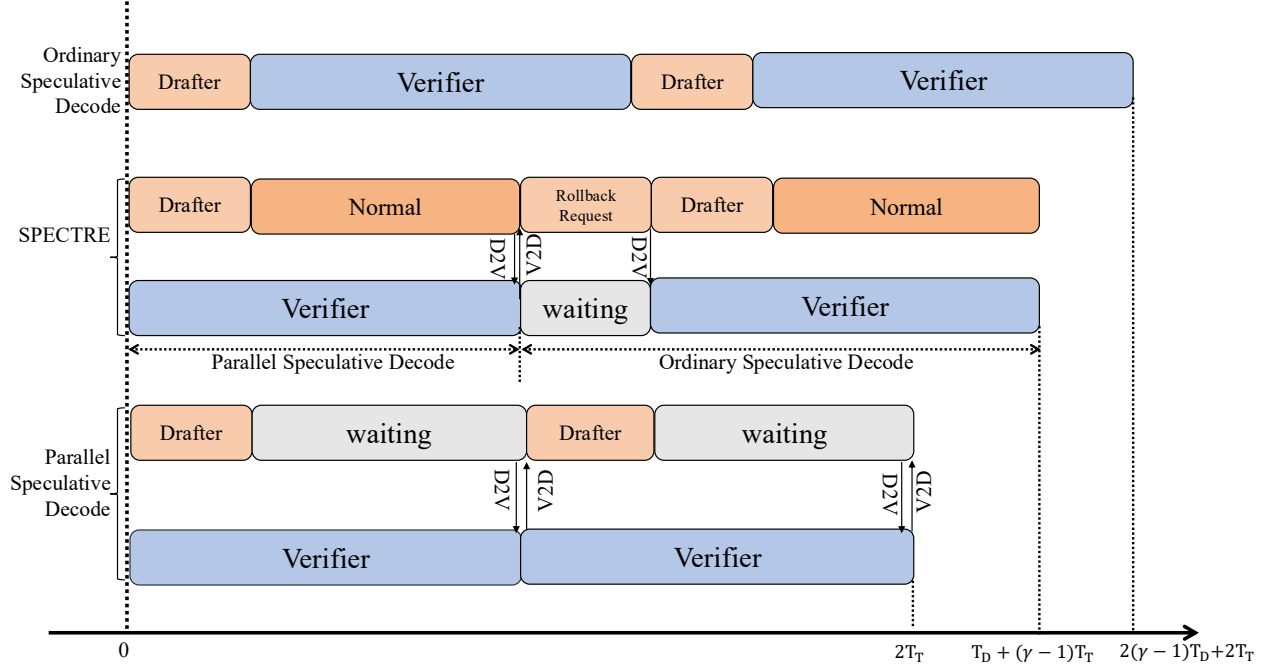


Figure 2 Timeline comparison of ordinary speculative decoding, parallel speculative decoding, and SPECTRE. Ordinary speculative decoding is fully serialized. Parallel speculative decoding overlaps draft generation and target verification. SPECTRE adaptively switches between parallel and ordinary coordination.

- We develop a speculative priority scheduling policy and a draft-side prompt compression method that preserve effective draft–target overlap and reduce draft latency, enabling tail-model services to support speculative drafting while continuing to serve their native workloads.
- We show through experiments across multiple draft–target model pairs, reasoning benchmarks, real-world long-context workloads, and a wide range of batch sizes that SPECTRE consistently improves large-model serving throughput while causing only minor interference to the native workloads of tail-model services.

2 Throughput-Guided Hybrid Coordination

Figure 2 illustrates the difference between ordinary and parallel speculative decoding. In ordinary speculative decoding, the target verification result provides the bonus token that seeds the next draft segment, so each round requires one target verification and only $\gamma - 1$ new draft steps:

$$\text{Thr}_{\text{ord}} = \frac{BL}{T_T + (\gamma - 1)T_D}, \quad (1)$$

where B is the batch size, L is the accepted length, γ is the number of speculative tokens per verification, and T_D , T_T denote the latency of one draft step and one target verification. Parallel speculative decoding instead prepares the next draft segment while the current segment is being verified. When $\gamma T_D < T_T$, draft generation can be hidden by target verification; however, if the current verification rejects the prefix needed by the pre-generated segment, the request falls back to autoregressive decoding. Let r be this fallback ratio. The parallel throughput is approximated as

$$\text{Thr}_{\text{par}} \approx \frac{B[r + (1 - r)L]}{T_T}. \quad (2)$$

Comparing the two throughputs gives a critical threshold

$$r^* = \frac{(\gamma - 1)LT_D}{(T_T + (\gamma - 1)T_D)(L - 1)}. \quad (3)$$

Parallel coordination is beneficial only when $r \leq r^*$; otherwise, fallback losses outweigh overlap benefits. SPECTRE therefore uses this threshold to switch between parallel and ordinary coordination at runtime, as shown in Fig. 2. The full derivation is provided in Appendix C.

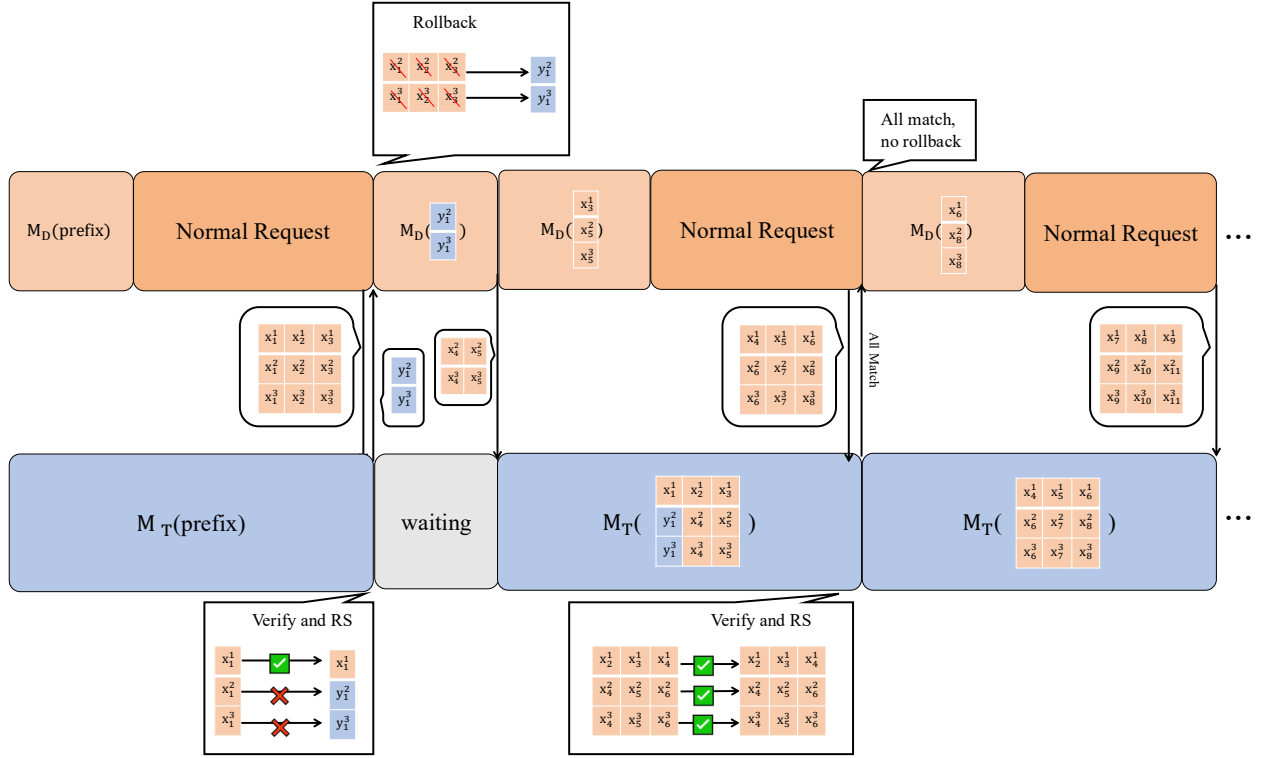


Figure 3 Illustration of SPECTRE decoding. The draft model (M_D) generates candidate tokens while serving normal requests, and the target model (M_T) verifies them via rejection sampling (RS). When the proportion of rollback requests becomes high, the system switches to ordinary coordination and the target waits for repaired draft tokens; otherwise, decoding proceeds without interruption with overlapped execution.

3 Method

We present **SPECTRE**, a hybrid ordinary-parallel speculative serving framework for remote draft-target deployment as shown in Figure 3. SPECTRE consists of a target server hosting the large model and a draft server hosting the lightweight draft model. At a high level, SPECTRE combines an target-side adaptive hybrid policy with a draft-side speculative priority scheduler to coordinate speculative execution under remote serving.

3.1 Target-Side Adaptive Hybrid Policy

At each decoding round, the target server selects the execution mode for the next round of the current target-side batch. The decision is made at the batch level and determines how the batch coordinates speculative verification and rollback handling in the next step.

Mode semantics. When a request’s previous speculative continuation can no longer be directly reused, it enters rollback, and the draft server must regenerate an updated continuation. SPECTRE supports two execution modes that differ in whether this draft-side rollback blocks target progress. In `ORDINARY` mode, the target launches the next verification pass only after the draft server completes this regeneration. In `PARALLEL` mode, the target continues its current computation while the draft server regenerates the updated continuation in parallel.

Rollback-Based Mode Selection. After the verification pass at round n , the target performs rejection sampling and identifies the rollback set $\mathcal{R}_n$. A request enters $\mathcal{R}_n$ if either its speculative prefix is rejected during target-side verification or its previously prepared draft continuation is invalidated by the verification result. The observed rollback ratio is then

$$\hat{r}_n = \frac{|\mathcal{R}_n|}{B}. \quad (4)$$

The target therefore determines the execution mode for round $n + 1$ according to

$$\text{mode}_{n+1} = \begin{cases} \text{PARALLEL}, & \hat{r}_n \leq r^*, \\ \text{ORDINARY}, & \hat{r}_n > r^*. \end{cases} \quad (5)$$

where r^* is the threshold derived from § 2.

Execution in ordinary mode. When $\text{mode}_n = \text{ORDINARY}$, the verification batch of round n is formed only after the rollback requests have been refreshed at the draft side. Let $\mathcal{R}_n$ and $\mathcal{S}_n$ denote the rollback and non-rollback requests at round n , respectively.

For each rollback request $i \in \mathcal{R}_n$, the target first sends the latest bonus token $b_n^{(i)}$ to the draft server. Conditioned on this updated prefix, the draft server performs $\gamma - 1$ auto-regressive decoding steps and returns a repaired continuation

$$\hat{\mathbf{d}}_n^{(i)} = [\hat{d}_{n,1}^{(i)}, \hat{d}_{n,2}^{(i)}, \dots, \hat{d}_{n,\gamma-1}^{(i)}]. \quad (6)$$

The target-side candidate sequence used for verification is then

$$\mathbf{y}_n^{(i)} = [b_n^{(i)}, \hat{\mathbf{d}}_n^{(i)}], \quad i \in \mathcal{R}_n. \quad (7)$$

For each non-rollback request $i \in \mathcal{S}_n$, no refresh is needed, and the target directly reuses the speculative continuation prepared in the previous round,

$$\bar{\mathbf{d}}_n^{(i)} = [\bar{d}_{n,1}^{(i)}, \bar{d}_{n,2}^{(i)}, \dots, \bar{d}_{n,\gamma}^{(i)}]. \quad (8)$$

Thus,

$$\mathbf{y}_n^{(i)} = \bar{\mathbf{d}}_n^{(i)}, \quad i \in \mathcal{S}_n. \quad (9)$$

The full verification batch is assembled as

$$\mathbf{Y}_n = \text{Merge}\left(\{\mathbf{y}_n^{(i)} : i \in \mathcal{R}_n\}, \{\mathbf{y}_n^{(i)} : i \in \mathcal{S}_n\}\right), \quad (10)$$

on which the target performs the verification pass for round n . Only after this refreshed batch has been formed and dispatched does the draft server resume speculative preparation for the next round.

Execution in parallel mode. When $\text{mode}_n = \text{PARALLEL}$, target-side verification and draft-side speculative generation proceed concurrently, without waiting for rollback refresh to complete.

For each non-rollback request $i \in \mathcal{S}_n$, the target reuses the cached speculative continuation from the previous round:

$$\mathbf{y}_n^{(i)} = \bar{\mathbf{d}}_n^{(i)} = [\bar{d}_{n,1}^{(i)}, \bar{d}_{n,2}^{(i)}, \dots, \bar{d}_{n,\gamma}^{(i)}], \quad i \in \mathcal{S}_n. \quad (11)$$

For each rollback request $i \in \mathcal{R}_n$, refreshed draft tokens are not yet available. The target therefore constructs a padded candidate sequence using the latest bonus token:

$$\mathbf{p}_n^{(i)} = \left[b_n^{(i)}, \underbrace{\emptyset, \emptyset, \dots, \emptyset}_{\gamma-1} \right], \quad i \in \mathcal{R}_n, \quad (12)$$

where $\emptyset$ denotes padding token with no draft prediction. The target-side candidate is

$$\mathbf{y}_n^{(i)} = \mathbf{p}_n^{(i)}, \quad i \in \mathcal{R}_n. \quad (13)$$

The mixed verification batch is then formed as

$$\mathbf{Y}_n = \text{Merge}\left(\{\mathbf{y}_n^{(i)} : i \in \mathcal{R}_n\}, \{\mathbf{y}_n^{(i)} : i \in \mathcal{S}_n\}\right), \quad (14)$$

and the target immediately performs the verification pass on $\mathbf{Y}_n$.

Meanwhile, the draft server prepares speculative continuations for the next round. To avoid ambiguity with the target-side candidates of the current round, we denote these next-round draft outputs by $\bar{\mathbf{d}}_{n+1}^{(i)}$. Specifically,

$$\bar{\mathbf{d}}_{n+1}^{(i)} = \begin{cases} [\bar{d}_{n+1,1}^{(i)}, \dots, \bar{d}_{n+1,\gamma}^{(i)}], & i \in \mathcal{R}_n, \\ [\bar{d}'_{n+1,1}^{(i)}, \dots, \bar{d}'_{n+1,\gamma}^{(i)}], & i \in \mathcal{S}_n, \end{cases} \quad (15)$$

where for $i \in \mathcal{R}_n$ generation is conditioned on the updated prefix $b_n^{(i)}$, while for $i \in \mathcal{S}_n$ generation continues from the previously cached speculative state. These continuations $\{\bar{\mathbf{d}}_{n+1}^{(i)}\}$ become available for verification in round $n+1$.

3.2 Draft-Side Optimization

speculative priority scheduling. Under remote deployment, the draft server serves both speculative and regular user requests, leading to increased latency under mixed batching. Let T_D^{mix} denote the per-step draft latency. If

$$\gamma T_D^{\text{mix}} > T_T, \quad (16)$$

the draft cannot produce γ tokens within one verification step, causing the target to stall. SPECTRE adopts a speculative priority, non-preemptive policy that prioritizes speculative requests in the next scheduling round while allowing the current round to complete. To prevent normal requests starvation, we enforce a lightweight fairness rule that schedules regular requests after every K speculative steps.

Draft-side context compression. When the target is accelerated by advanced inference techniques such as tensor parallelism (TP) [7], draft-side speculative generation may become the runtime bottleneck, weakening draft-target overlap. SPECTRE addresses this case by using a streamingLLM-style[8] compression on draft-side context during prefill. Specifically, given an input sequence $x_{1:S}$ where S is sequence length, the draft retains only a prefix and a suffix,

$$\mathcal{E}(x_{1:S}) = [x_{:\lfloor \frac{p}{2} S \rfloor}, x_{-\lfloor \frac{p}{2} S \rfloor+1:}], \quad (17)$$

where p is the retained ratio. This compression reduces the effective context length seen by the draft and thereby lowers speculative-generation latency. The resulting context mismatch between draft and target may

reduce the accepted length, but can still improve overall target-side throughput when draft latency dominates the critical path.

Despite these optimizations, fully satisfying Eq. (16) remains challenging in practical serving environments due to system-level variability and workload dynamics. As a result, transient violations of this condition may still occur in certain cases. When such violations happen, the system falls back to a conservative execution mode in which the target waits for the completion of all speculative token generation before verification.

4 Experiments

4.1 Experimental Setup

Models and deployment. We evaluate SPECTRE on three draft–target model pairs: Qwen3-0.6B (TP1) → Qwen3-32B (TP1), Qwen3-0.6B (TP1) → Qwen3-235B-A22B (TP8) [9], and DeepSeek-R1-Distill-Qwen-1.5B (TP1) → DeepSeek-R1-Distill-Qwen-32B (TP1) [10]. The draft and target models are deployed on separate H200 GPUs[11] and communicate remotely through our serving framework. Additional implementation details are provided in Appendix A.

Datasets. We evaluate SPECTRE on six datasets: GSM8K [12], MATH500 [13], Minerva Math, LongBench [14, 15], MRCR [16], and ShareGPT. These datasets cover grade-school math, advanced mathematical reasoning, STEM-oriented problem solving, long-context question answering and retrieval, and conversational data.

Baselines. We compare SPECTRE against autoregressive decoding (AR), Standalone [17], EAGLE-3 [18], PEARL [6], and MineDraft [19], under the same target model, decoding configuration, batch size, and hardware budget. For the Qwen3 series, we use publicly available EAGLE-3 checkpoints¹. We do not report EAGLE-3 results for the DeepSeek series because no public checkpoints are available. For MineDraft, we use the official open-source implementation.² Unless otherwise specified, we use greedy decoding (temperature = 0), and the target verifies four speculative tokens per round. Detailed settings for each experiment are provided in Appendix E.

4.2 TP1 Target Throughput Speedup

Table 1 reports target throughput speedups for TP1 targets on reasoning datasets (GSM8K, MATH500, and Minerva Math) and long-context datasets (ShareGPT and LongBench) at batch size = 32, 64, 128. SPECTRE achieves the highest throughput in all reported settings, with up to **2.28× speedup** over autoregressive decoding. Compared with prior speculative decoding baselines, SPECTRE delivers more consistent gains across batch sizes and datasets. EAGLE3 and PEARL often lose speedup as batch size increases and can even underperform autoregressive decoding on long-context datasets. MINE DRAFT is less stable and is excluded from long-context experiments because of out-of-memory issues. In contrast, SPECTRE remains effective across the full set of reported TP1 results. The Δ rows further show that SPECTRE consistently outperforms the SOTA method in each setting, with up to an additional **66%** gain on LongBench. Overall, these results show that SPECTRE consistently improves target throughput across both reasoning and long-context datasets. Moreover, we illustrate SPECTRE at small batch size in Appendix D.1.

4.3 TP8 Target Throughput Speedup

Under TP=8 target deployment, target-side verification becomes much faster relative to draft generation, making parallel speculative decoding more difficult to sustain. We therefore enable context compression and set $p = 0.1$ in (17). Figure 4 reports throughput at batch size 128 on LongBench-v2, MRCR, and

¹https://huggingface.co/AngelSlim/Qwen3-32B_eagle3 and <https://huggingface.co/lmsys/Qwen3-235B-A22B-EAGLE3>

²The official MineDraft implementation is based on vLLM 0.9.2[20] and performs relatively poorly in our setup, sometimes even underperforming autoregressive decoding in `sglang` v0.5.7[21]. We attribute this to implementation limitations rather than to the underlying method itself. In addition, MineDraft frequently encounters out-of-memory (OOM) issues under long-context settings, so we exclude it from long-context experiments.

Model (bs)	Method	GSM8K		Math500		Minerva Math		ShareGPT		LongBench	
		Tok/s	Speedup	Tok/s	Speedup	Tok/s	Speedup	Tok/s	Speedup	Tok/s	Speedup
Qwen3-32B (bs=32)	AR	1009.48	1.00×	1001.40	1.00×	1003.11	1.00×	753.44	1.00×	308.06	1.00×
	EAGLE3	1380.38	1.37×	1436.39	1.43×	1283.62	1.28×	878.34	1.17×	248.30	0.81×
	PEARL	1255.99	1.24×	1020.93	1.02×	1241.92	1.23×	945.24	1.25×	<u>532.38</u>	1.73×
	MineDraft	759.50	0.75×	832.34	0.83×	782.61	0.78×	OOM	OOM	OOM	OOM
	Standalone	1559.03	1.54×	1594.78	1.59×	1588.71	1.58×	993.60	1.32×	488.57	1.59×
	SPECTRE	1764.78	1.75×	1728.68	1.73×	1776.95	1.77×	1185.06	1.57×	653.69	2.12×
	Δ (%)		↑ 13.6%		↑ 8.8%		↑ 12.0%		↑ 18.9%		↑ 22.5%
Qwen3-32B (bs=64)	AR	1432.60	1.00×	1423.07	1.00×	1422.39	1.00×	806.95	1.00×	310.59	1.00×
	EAGLE3	1782.63	1.24×	1797.62	1.26×	1776.18	1.25×	959.11	1.19×	235.85	0.76×
	PEARL	1483.45	1.04×	754.85	0.53×	828.45	0.58×	<u>1038.49</u>	1.29×	495.32	1.59×
	MineDraft	1193.47	0.83×	1295.63	0.91×	1231.83	0.87×	OOM	OOM	OOM	OOM
	Standalone	1923.28	1.34×	1945.95	1.37×	1924.57	1.35×	1015.15	1.26×	517.73	1.67×
	SPECTRE	2183.14	1.52×	2148.39	1.51×	2152.47	1.51×	1353.11	1.68×	706.81	2.28×
	Δ (%)		↑ 13.4%		↑ 10.2%		↑ 11.9%		↑ 30.2%		↑ 36.5%
Qwen3-32B (bs=128)	AR	1762.27	1.00×	1762.52	1.00×	1590.46	1.00×	789.93	1.00×	465.29	1.00×
	EAGLE3	1930.57	1.10×	1941.57	1.10×	1774.24	1.12×	982.93	1.24×	302.95	0.65×
	PEARL	893.30	0.51×	637.58	0.36×	842.30	0.53×	<u>1069.28</u>	1.35×	483.91	1.04×
	MineDraft	1556.27	0.88×	1544.90	0.88×	1409.60	0.89×	OOM	OOM	OOM	OOM
	Standalone	2037.20	1.16×	2053.80	1.17×	1951.21	1.23×	1048.48	1.33×	515.68	1.11×
	SPECTRE	2430.48	1.38×	2421.28	1.37×	2348.38	1.48×	1359.09	1.72×	800.45	1.72×
	Δ (%)		↑ 19.0%		↑ 17.1%		↑ 20.3%		↑ 27.4%		↑ 55.0%
DS-32B (bs=32)	AR	1025.18	1.00×	1017.38	1.00×	1014.42	1.00×	800.39	1.00×	449.11	1.00×
	PEARL	1407.74	1.37×	1403.23	1.38×	1429.11	1.41×	456.09	0.57×	202.83	0.45×
	MineDraft	957.13	0.93×	1003.12	0.99×	992.49	0.98×	OOM	OOM	OOM	OOM
	Standalone	2137.28	2.08×	2147.52	2.11×	2131.65	2.10×	925.76	1.16×	565.95	1.26×
	SPECTRE	2268.75	2.21×	2259.68	2.22×	2271.10	2.24×	1408.17	1.76×	833.92	1.86×
	Δ (%)		↑ 6.2%		↑ 5.2%		↑ 6.7%		↑ 52.1%		↑ 47.6%
DS-32B (bs=64)	AR	1431.09	1.00×	1428.67	1.00×	1412.98	1.00×	841.87	1.00×	478.90	1.00×
	PEARL	1681.16	1.17×	1050.97	0.74×	988.95	0.70×	449.22	0.53×	197.65	0.41×
	MineDraft	1462.13	1.02×	1475.78	1.03×	1465.81	1.04×	OOM	OOM	OOM	OOM
	Standalone	2741.32	1.92×	2147.52	1.50×	2753.48	1.95×	954.79	1.13×	592.64	1.24×
	SPECTRE	2806.23	1.96×	2884.79	2.02×	2867.57	2.03×	1274.28	1.50×	887.01	1.85×
	Δ (%)		↑ 2.1%		↑ 34.7%		↑ 4.1%		↑ 32.7%		↑ 49.2%
DS-32B (bs=128)	AR	1766.62	1.00×	1758.49	1.00×	1584.39	1.00×	826.19	1.00×	480.42	1.00×
	PEARL	911.46	0.52×	637.67	0.36×	950.51	0.60×	937.30	1.13×	218.80	0.46×
	MineDraft	1625.18	0.92×	1621.92	0.92×	1552.44	0.98×	OOM	OOM	OOM	OOM
	Standalone	2885.66	1.63×	2906.47	1.65×	2753.25	1.74×	988.23	1.20×	599.22	1.25×
	SPECTRE	3131.47	1.77×	3218.42	1.83×	3029.84	1.91×	1489.56	1.80×	998.38	2.08×
	Δ (%)		↑ 8.6%		↑ 10.9%		↑ 9.8%		↑ 50.0%		↑ 66.4%

Table 1 Throughput (Tok/s) and speedup over AR for each model, batch size, and benchmark. For each benchmark, The best results are marked in bold and the SOTA results are underlined for each benchmark. Light yellow rows correspond to SPECTRE, and light blue rows show SPECTRE’s relative speedup gain over the SOTA method. Missing MineDraft entries indicate out-of-memory (OOM) errors. The detail of experiment settings is shown in E.1.

ShareGPT. SPECTRE consistently achieves the best throughput across all three datasets, exceeding the strongest competing method by **19%**, **12%**, and **14%**, respectively. In contrast, existing parallel speculative decoding methods struggle in this regime: EAGLE3 and STANDALONE provide only limited gains, while PEARL even underperforms the autoregressive baseline. PEARL degradation stems from the asymmetric compute allocation in our setting, where the target model is deployed with TP=8 while the draft model uses only a single GPU, causing draft generation to become the system bottleneck and preventing effective overlap with target-side verification. Overall, these results highlight that naive parallelization is insufficient under high-concurrency and imbalanced resource settings, whereas SPECTRE remains robust and continues to deliver substantial throughput improvements.

Model Pair	AR	EAGLE3	PEARL	Standalone	SPECTRE
Qwen3-32B / Qwen3-0.6B	2.33	2.52	1.60	2.62	2.71 (+1.16×)
Qwen3-235B-A22B / Qwen3-0.6B	0.38	0.47	0.16	0.46	0.48 (+1.26×)
DeepSeek-R1-Distill-Qwen-32B / DeepSeek-R1-Distill-Qwen-1.5B	0.37	–	0.42	0.44	0.67 (+1.81×)

Table 2 System-level benefit efficiency measured in dollar/1000s/GPU (higher is better). For SPECTRE, the values in parentheses denote the relative improvement over AR. SPECTRE consistently achieves the highest benefit efficiency across all model configurations. Price from <https://help.aliyun.com/zh/model-studio/model-pricing>. The detail of experiment settings is shown in E.4.

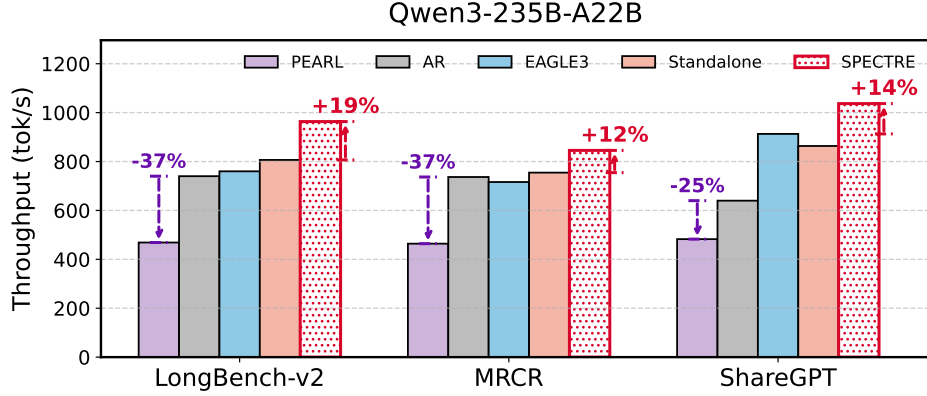


Figure 4 Throughput comparison under high-concurrency settings (batch size=128) using Qwen3-235B-A22B (TP=8) as the target model and Qwen3-0.6B (TP=1) as the draft model. We evaluate across three workloads: LongBench-v2, MRCR, and ShareGPT. SPECTRE consistently achieves the highest throughput, outperforming all baselines. The red upward arrows indicate the relative improvement of SPECTRE over the strongest baseline (Standalone for LongBench-v2 and MRCR, and EAGLE3 for ShareGPT), while the purple downward arrows indicate the relative degradation of PEARL compared to the autoregressive baseline (AR). Detailed experimental settings are provided in E.2.

4.4 Performance under shared draft traffic

We next evaluate SPECTRE in a multi-tenant setting where the draft server simultaneously serves background requests and speculative decoding traffic. As shown in Fig. 5, moderate draft-side load has little impact on target throughput. Draft throughput remains nearly unchanged up to 8 QPS and degrades only under heavier load, while target throughput decreases by less than 2% even at draft-QPS 8 and target-QPS 32 for both Qwen3-32B and DeepSeek-R1-Distill-Qwen-32B. Table 2 further shows that SPECTRE still delivers the highest system-level benefit in this shared-serving setting, improving from 0.38 to 0.48 (1.26 $\times$) on Qwen3-235B-A22B and from 0.37 to 0.67 (1.81 $\times$) on DeepSeek-R1-Distill-Qwen-32B. Although draft throughput drops slightly under high load, the overall benefit continues to increase, indicating that SPECTRE can effectively translate additional draft-side utilization into higher end-to-end efficiency. In contrast, PEARL achieves limited gains in this setting, as its draft model occupies a dedicated GPU but does not serve regular user traffic, leading to lower overall resource utilization and reduced system-level benefit.

4.5 Accepted Length Analysis

Table 3 reports mean accepted length averaged over batch sizes. We compare SPECTRE with EAGLE3, PEARL, and STANDALONE. We exclude MINE DRAFT because it changes batch-level scheduling but not token acceptance, and therefore has the same accepted length as STANDALONE. Across all datasets and models, STANDALONE achieves the largest accepted length, as expected from ordinary speculative decoding without parallel interference. SPECTRE yields accepted lengths that remain close to STANDALONE, for example, 2.59–3.11 versus 3.13–3.22 on Qwen3-32B, and 3.00–3.61 versus 3.41–3.77 on DeepSeek-R1-Distill-Qwen-32B. By contrast, EAGLE3 and PEARL show noticeably shorter accepted lengths across settings. Together with Table 1, these results show that SPECTRE attains higher throughput than prior parallel baselines while maintaining accepted lengths closer to STANDALONE.

5 Conclusion

We present **SPECTRE**, a hybrid ordinary-parallel speculative decoding framework that improves the efficiency of LLM inference by addressing system-level inefficiencies in existing approaches. Experiments across multiple benchmarks show that SPECTRE consistently achieves the highest throughput, delivering up to 2.28 $\times$ speedup over AR and up to 66% improvement over STANDALONE, while maintaining acceptance lengths close to the latter. These results suggest that parallel speculative decode alone is insufficient for high performance, and that jointly considering acceptance behavior and execution efficiency is critical. Overall, SPECTRE provides an effective direction for scalable and efficient LLM inference.

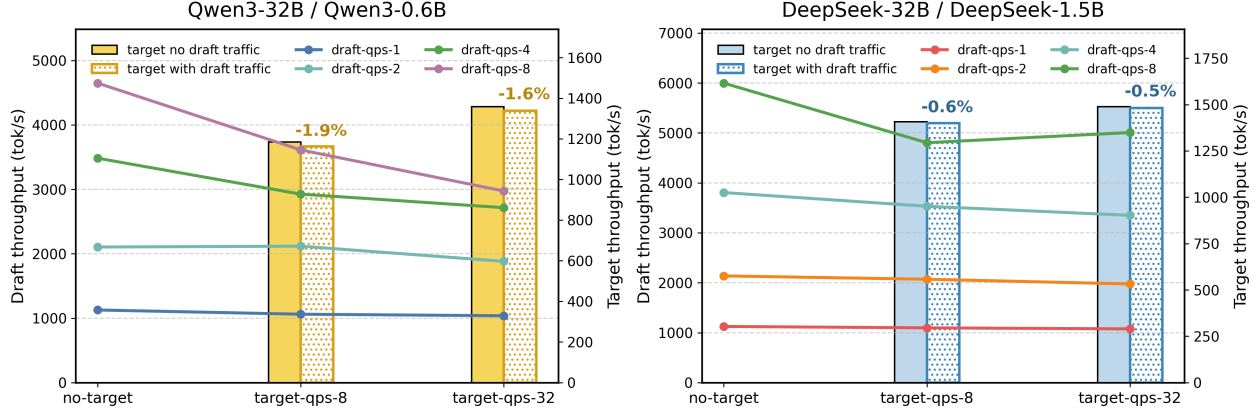


Figure 5 Throughput under mixed draft-side traffic. The draft model serves both speculative requests from the target model and background user requests (query per second (QPS), denoted by draft-qps- k). The x-axis shows different target request rates. Solid bars denote target throughput without background draft traffic, while hatched bars denote target throughput with background traffic. Lines represent draft throughput under different background loads. The annotated percentages indicate the relative decrease in target throughput when background traffic is present. Detailed experimental settings are provided in E.3.

Dataset	AR	EAGLE3	PEARL	Standalone	SPECTRE
Qwen3-32B					
GSM8K	1.00	1.96	1.96	3.17	2.61
Math500	1.00	1.96	1.66	3.18	2.61
Minerva Math	1.00	1.90	1.75	3.18	2.59
ShareGPT	1.00	2.44	2.03	3.13	3.11
Longbench	1.00	1.90	2.05	3.22	2.68
DeepSeek-R1-Distill-Qwen-32B					
GSM8K	1.00	—	2.19	3.41	3.00
Math500	1.00	—	2.01	3.47	3.07
Minerva Math	1.00	—	1.99	3.47	3.02
ShareGPT	1.00	—	2.16	3.57	3.27
Longbench	1.00	—	1.91	3.77	3.61

Table 3 Mean accepted length averaged over batch sizes {32, 64, 128}. Detailed experimental settings are provided in E.1.

References

- [1] Amey Agrawal, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S. Gulavani, and Ramachandran Ramjee. Sarathi: Efficient llm inference by piggybacking decodes with chunked prefills. *ArXiv*, abs/2308.16369, 2023. URL <https://api.semanticscholar.org/CorpusID:261395577>.
- [2] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. Splitwise: Efficient generative llm inference using phase splitting. In *Proceedings of the 51st Annual International Symposium on Computer Architecture, ISCA '24*, pages 118–132. IEEE Press, 2025. ISBN 9798350326581. doi: 10.1109/ISCA59077.2024.00019. URL <https://doi.org/10.1109/ISCA59077.2024.00019>.
- [3] Seungbeom Choi, Sunho Lee, Yeonjae Kim, Jongse Park, Youngjin Kwon, and Jaehyuk Huh. Serving heterogeneous machine learning models on Multi-GPU servers with Spatio-Temporal sharing. In *2022 USENIX Annual Technical Conference (USENIX ATC 22)*, pages 199–216, Carlsbad, CA, July 2022. USENIX Association. ISBN 978-1-939133-29-53. URL <https://www.usenix.org/conference/atc22/presentation/choi-seungbeom>.
- [4] Yuxing Xiang, Xue Li, Kun Qian, Yufan Yang, Diwen Zhu, Wenyan Yu, Ennan Zhai, Xuanzhe Liu, Xin Jin, and Jingren Zhou. Aegaeon: Effective gpu pooling for concurrent llm serving on the market. In *Proceedings of*

- the ACM SIGOPS 31st Symposium on Operating Systems Principles, SOSP '25, pages 1030–1045, New York, NY, USA, 2025. Association for Computing Machinery. ISBN 9798400718700. doi: 10.1145/3731569.3764815. URL <https://doi.org/10.1145/3731569.3764815>.
- [5] Yuhao Shen, Tianyu Liu, Junyi Shen, Jinyang Wu, Quan Kong, Li Huan, and Cong Wang. Double: Breaking the acceleration limit via double retrieval speculative parallelism, 2026. URL <https://arxiv.org/abs/2601.05524>.
 - [6] Tianyu Liu, Yun Li, Qitan Lv, Kai Liu, Jianchen Zhu, Winston Hu, and Xiao Sun. PEARL: Parallel speculative decoding with adaptive draft length. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=QOXrVMiHGK>.
 - [7] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-lm: Training multi-billion parameter language models using model parallelism, 2020. URL <https://arxiv.org/abs/1909.08053>.
 - [8] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=NG7sS51zVF>.
 - [9] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jing Zhou, Jingren Zhou, Junyang Lin, Kai Dang, Keqin Bao, Kexin Yang, Le Yu, Lianghao Deng, Mei Li, Mingfeng Xue, Mingze Li, Pei Zhang, Peng Wang, Qin Zhu, Rui Men, Ruize Gao, Shixuan Liu, Shuang Luo, Tianhao Li, Tianyi Tang, Wenbiao Yin, Xingzhang Ren, Xinyu Wang, Xinyu Zhang, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yinger Zhang, Yu Wan, Yuqiong Liu, Zekun Wang, Zeyu Cui, Zhenru Zhang, Zhipeng Zhou, and Zihan Qiu. Qwen3 technical report, 2025. URL <https://arxiv.org/abs/2505.09388>.
 - [10] DeepSeek-AI, Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, Damai Dai, Daya Guo, Dejian Yang, Deli Chen, Dongjie Ji, Erhang Li, Fangyun Lin, Fucong Dai, Fuli Luo, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Han Bao, Hanwei Xu, Haocheng Wang, Haowei Zhang, Honghui Ding, Huajian Xin, Huazuo Gao, Hui Li, Hui Qu, J. L. Cai, Jian Liang, Jianzhong Guo, Jiaqi Ni, Jiashi Li, Jiawei Wang, Jin Chen, Jingchang Chen, Jingyuan Yuan, Junjie Qiu, Junlong Li, Junxiao Song, Kai Dong, Kai Hu, Kaige Gao, Kang Guan, Kexin Huang, Kuai Yu, Lean Wang, Lecong Zhang, Lei Xu, Leyi Xia, Liang Zhao, Litong Wang, Liyue Zhang, Meng Li, Miaojun Wang, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Mingming Li, Ning Tian, Panpan Huang, Peiyi Wang, Peng Zhang, Qiancheng Wang, Qihao Zhu, Qinyu Chen, Qiushi Du, R. J. Chen, R. L. Jin, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, Runxin Xu, Ruoyu Zhang, Ruyi Chen, S. S. Li, Shanghao Lu, Shangyan Zhou, Shanhua Chen, Shaoqing Wu, Shengfeng Ye, Shengfeng Ye, Shirong Ma, Shiyu Wang, Shuang Zhou, Shuiping Yu, Shunfeng Zhou, Shuting Pan, T. Wang, Tao Yun, Tian Pei, Tianyu Sun, W. L. Xiao, Wangding Zeng, Wanbiao Zhao, Wei An, Wen Liu, Wenfeng Liang, Wenjun Gao, Wenqin Yu, Wentao Zhang, X. Q. Li, Xiangyue Jin, Xianzu Wang, Xiao Bi, Xiaodong Liu, Xiaohan Wang, Xiaojin Shen, Xiaokang Chen, Xiaokang Zhang, Xiaosha Chen, Xiaotao Nie, Xiaowen Sun, Xiaoxiang Wang, Xin Cheng, Xin Liu, Xin Xie, Xingchao Liu, Xingkai Yu, Xinnan Song, Xinxia Shan, Xinyi Zhou, Xinyu Yang, Xinyuan Li, Xuecheng Su, Xuheng Lin, Y. K. Li, Y. Q. Wang, Y. X. Wei, Y. X. Zhu, Yang Zhang, Yanhong Xu, Yanhong Xu, Yanping Huang, Yao Li, Yao Zhao, Yaofeng Sun, Yaohui Li, Yaohui Wang, Yi Yu, Yi Zheng, Yichao Zhang, Yifan Shi, Yiliang Xiong, Ying He, Ying Tang, Yishi Piao, Yisong Wang, Yixuan Tan, Yiyang Ma, Yiyuan Liu, Yongqiang Guo, Yu Wu, Yuan Ou, Yuchen Zhu, Yudian Wang, Yue Gong, Yuheng Zou, Yujia He, Yukun Zha, Yunfan Xiong, Yunxian Ma, Yuting Yan, Yuxiang Luo, Yuxiang You, Yuxuan Liu, Yuyang Zhou, Z. F. Wu, Z. Z. Ren, Zehui Ren, Zhangli Sha, Zhe Fu, Zhean Xu, Zhen Huang, Zhen Zhang, Zhenda Xie, Zhengyan Zhang, Zhewen Hao, Zhibin Gou, Zhicheng Ma, Zhigang Yan, Zhihong Shao, Zhipeng Xu, Zhiyu Wu, Zhongyu Zhang, Zhuoshu Li, Zihui Gu, Zijia Zhu, Zijun Liu, Zilin Li, Ziwei Xie, Ziyang Song, Ziyi Gao, and Zizheng Pan. Deepseek-v3 technical report, 2025. URL <https://arxiv.org/abs/2412.19437>.
 - [11] NVIDIA. NVIDIA H200 GPU. <https://www.nvidia.com/en-us/data-center/h200/>, 2023.
 - [12] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems, 2021. URL <https://arxiv.org/abs/2110.14168>.
 - [13] Hunter Lightman, Vineet Kosaraju, Yuri Burda, Harrison Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let’s verify step by step. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=v8LOpN6EOi>.

- [14] Yushi Bai, Xin Lv, Jiajie Zhang, Hongchang Lyu, Jiankai Tang, Zhidian Huang, Zhengxiao Du, Xiao Liu, Aohan Zeng, Lei Hou, Yuxiao Dong, Jie Tang, and Juanzi Li. LongBench: A bilingual, multitask benchmark for long context understanding. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar, editors, *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 3119–3137, Bangkok, Thailand, August 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.acl-long.172. URL <https://aclanthology.org/2024.acl-long.172/>.
- [15] Yushi Bai, Shangqing Tu, Jiajie Zhang, Hao Peng, Xiaozhi Wang, Xin Lv, Shulin Cao, Jiazheng Xu, Lei Hou, Yuxiao Dong, Jie Tang, and Juanzi Li. LongBench v2: Towards deeper understanding and reasoning on realistic long-context multitasks. In Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar, editors, *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 3639–3664, Vienna, Austria, July 2025. Association for Computational Linguistics. ISBN 979-8-89176-251-0. doi: 10.18653/v1/2025.acl-long.183. URL <https://aclanthology.org/2025.acl-long.183/>.
- [16] Kiran Vodrahalli, Santiago Ontanon, Nilesch Tripuraneni, Kelvin Xu, Sanil Jain, Rakesh Shivanna, Jeffrey Hui, Nishanth Dikkala, Mehran Kazemi, Bahare Fatemi, Rohan Anil, Ethan Dyer, Siamak Shakeri, Roopali Vij, Harsh Mehta, Vinay Ramasesh, Quoc Le, Ed Chi, Yifeng Lu, Orhan Firat, Angeliki Lazaridou, Jean-Baptiste Lespiau, Nithya Attaluri, and Kate Olszewska. Michelangelo: Long context evaluations beyond haystacks via latent structure queries, 2024. URL <https://arxiv.org/abs/2409.12640>.
- [17] Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *Proceedings of the 40th International Conference on Machine Learning, ICML’23*. JMLR.org, 2023.
- [18] Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. EAGLE-3: Scaling up inference acceleration of large language models via training-time test. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025. URL <https://openreview.net/forum?id=4exx1hUffq>.
- [19] Zhenwei Tang, Arun Verma, Zijian Zhou, Zhaoxuan Wu, Alok Prakash, Daniela Rus, and Bryan Kian Hsiang Low. Minedraft: A framework for batch parallel speculative decoding, 2026. URL <https://arxiv.org/abs/2603.18016>.
- [20] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th Symposium on Operating Systems Principles, SOSP ’23*, pages 611–626, New York, NY, USA, 2023. Association for Computing Machinery. ISBN 9798400702297. doi: 10.1145/3600006.3613165. URL <https://doi.org/10.1145/3600006.3613165>.
- [21] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. Sglang: efficient execution of structured language model programs. In *Proceedings of the 38th International Conference on Neural Information Processing Systems, NIPS ’24*, Red Hook, NY, USA, 2024. Curran Associates Inc. ISBN 9798331314385.
- [22] Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. Accelerating large language model decoding with speculative sampling, 2023. URL <https://arxiv.org/abs/2302.01318>.
- [23] Heming Xia, Tao Ge, Peiyi Wang, Si-Qing Chen, Furu Wei, and Zhifang Sui. Speculative decoding: Exploiting speculative execution for accelerating seq2seq generation. In Houda Bouamor, Juan Pino, and Kalika Bali, editors, *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 3909–3925, Singapore, December 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.findings-emnlp.257. URL <https://aclanthology.org/2023.findings-emnlp.257/>.
- [24] Minghao Yan, Saurabh Agarwal, and Shivaram Venkataraman. Decoding speculative decoding. In Luis Chiruzzo, Alan Ritter, and Lu Wang, editors, *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 6460–6473, Albuquerque, New Mexico, April 2025. Association for Computational Linguistics. ISBN 979-8-89176-189-6. doi: 10.18653/v1/2025.naacl-long.328. URL <https://aclanthology.org/2025.naacl-long.328/>.
- [25] Yinrong Hong, Zhiqian Tan, and Kai Hu. Inference-cost-aware dynamic tree construction for efficient inference in large language models. In *The Fourteenth International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=iaWyRYthFf>.
- [26] Zihong Zhang, Zuchao Li, Lefei Zhang, Ping Wang, and Hai Zhao. Racer: Retrieval-augmented contextual rapid speculative decoding, 2026. URL <https://arxiv.org/abs/2604.14885>.

- [27] Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. Eagle: speculative sampling requires rethinking feature uncertainty. In Proceedings of the 41st International Conference on Machine Learning, ICML'24. JMLR.org, 2024.
- [28] Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. EAGLE-2: Faster inference of language models with dynamic draft trees. In Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen, editors, Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing, pages 7421–7432, Miami, Florida, USA, nov 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.emnlp-main.422. URL <https://aclanthology.org/2024.emnlp-main.422/>.
- [29] Xupeng Miao, Gabriele Oliaro, Zhihao Zhang, Xinhao Cheng, Zeyu Wang, Zhengxin Zhang, Rae Ying Yee Wong, Alan Zhu, Lijie Yang, Xiaoxiang Shi, Chunan Shi, Zhuoming Chen, Daiyaan Arfeen, Reyna Abhyankar, and Zhihao Jia. Specinfer: Accelerating large language model serving with tree-based speculative inference and verification. In Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3, ASPLOS '24, pages 932–949. ACM, April 2024. doi: 10.1145/3620666.3651335. URL <http://dx.doi.org/10.1145/3620666.3651335>.
- [30] Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D. Lee, Deming Chen, and Tri Dao. Medusa: Simple llm inference acceleration framework with multiple decoding heads. In Proceedings of the 41st International Conference on Machine Learning, ICML'24. JMLR.org, 2024.
- [31] Tanishq Kumar, Tri Dao, and Avner May. Speculative speculative decoding. In The Fourteenth International Conference on Learning Representations, 2026. URL <https://openreview.net/forum?id=aL1Wnml9Ef>.
- [32] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. Distserve: disaggregating prefill and decoding for goodput-optimized large language model serving. In Proceedings of the 18th USENIX Conference on Operating Systems Design and Implementation, OSDI'24, USA, 2024. USENIX Association. ISBN 978-1-939133-40-3.
- [33] Biao Sun, Ziming Huang, Hanyu Zhao, Wencong Xiao, Xinyi Zhang, Yong Li, and Wei Lin. Llumnix: dynamic scheduling for large language model serving. In Proceedings of the 18th USENIX Conference on Operating Systems Design and Implementation, OSDI'24, USA, 2024. USENIX Association. ISBN 978-1-939133-40-3.
- [34] Jingqi Feng, Yukai Huang, Rui Zhang, Sicheng Liang, Ming Yan, and Jie Wu. Windserve: Efficient phase-disaggregated llm serving with stream-based dynamic scheduling. In Proceedings of the 52nd Annual International Symposium on Computer Architecture, ISCA '25, pages 1283–1295, New York, NY, USA, 2025. Association for Computing Machinery. ISBN 9798400712616. doi: 10.1145/3695053.3730999. URL <https://doi.org/10.1145/3695053.3730999>.
- [35] Zejia Lin, Hongxin Xu, Guanyi Chen, Zhiguang Chen, Yutong Lu, and Xianwei Zhang. Bullet: Boosting gpu utilization for llm serving via dynamic spatial-temporal orchestration. In Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2, ASPLOS '26, pages 290–306, New York, NY, USA, 2026. Association for Computing Machinery. ISBN 9798400723599. doi: 10.1145/3779212.3790135. URL <https://doi.org/10.1145/3779212.3790135>.
- [36] Jiangfei Duan, Runyu Lu, Haojie Duanmu, Xiuhong Li, Xingcheng Zhang, Dahua Lin, Ion Stoica, and Hao Zhang. Muxserve: flexible spatial-temporal multiplexing for multiple llm serving. In Proceedings of the 41st International Conference on Machine Learning, ICML'24. JMLR.org, 2024.
- [37] Ruoyu Qin, Zheming Li, Weiran He, Jialei Cui, Heyi Tang, Feng Ren, Teng Ma, Shangming Cai, Yineng Zhang, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. Mooncake: A kvcache-centric disaggregated architecture for llm serving. ACM Trans. Storage, November 2025. ISSN 1553-3077. doi: 10.1145/3773772. URL <https://doi.org/10.1145/3773772>. Just Accepted.
- [38] Wonbeom Lee, Jungi Lee, Junghwan Seo, and Jaewoong Sim. Infinigen: efficient generative inference of large language models with dynamic kv cache management. In Proceedings of the 18th USENIX Conference on Operating Systems Design and Implementation, OSDI'24, USA, 2024. USENIX Association. ISBN 978-1-939133-40-3.
- [39] Shiwei Gao, Qing Wang, Shaoxun Zeng, Youyou Lu, and Jiwu Shu. Weaver: efficient multi-llm serving with attention offloading. In Proceedings of the 2025 USENIX Conference on Usenix Annual Technical Conference, USENIX ATC '25, USA, 2025. USENIX Association. ISBN 978-1-939133-48-9.

- [40] Shan Yu, Jiarong Xing, Yifan Qiao, Mingyuan Ma, Yangmin Li, Yang Wang, Shuo Yang, Zhiqiang Xie, Shiyi Cao, Ke Bao, Ion Stoica, Harry Xu, and Ying Sheng. Prism: Unleashing gpu sharing for cost-efficient multi-llm serving, 2025. URL <https://arxiv.org/abs/2505.04021>.

A Implementation Details

System overview. SPECTRE separates speculative decoding into two cooperating services: a target server and a draft server. The target server is responsible for maintaining the authoritative generation state and for performing verification with the large model. The draft server runs asynchronously and predicts future tokens that may be accepted by the target. The two servers exchange compact synchronization messages over a lightweight messaging layer. Each message is tagged with both a request identifier and a decoding-round identifier so that delayed or duplicated messages can be safely ignored.

A.1 Target side

The target server maintains the authoritative decoding state for every request and is the only component allowed to commit output target tokens. For request i at round n , let $\mathbf{p}_n^{(i)}$ denote the committed prefix, and let $\mathbf{d}_n^{(i)}$ denote the speculative tokens currently available at the target. The target then forms a verification candidate

$$\mathbf{y}_n^{(i)} = \text{Assemble}\left(\mathbf{p}_n^{(i)}, \mathbf{d}_n^{(i)}\right), \quad (18)$$

and submits the batch

$$\mathbf{Y}_n = \{\mathbf{y}_n^{(i)}\}_{i=1}^B \quad (19)$$

to the target model for verification.

Meanwhile, the target sends speculative-generation requests to the remote draft server. Let

$$\mathbf{q}_n^{(i)} = (i, n, \mathbf{p}_n^{(i)}) \quad (20)$$

denote the draft query issued for request i at round n , where (i, n) is the request-round identifier. The draft server returns a speculative tokens

$$\tilde{\mathbf{d}}_n^{(i)} = [\tilde{d}_{n,1}^{(i)}, \dots, \tilde{d}_{n,\gamma}^{(i)}]. \quad (21)$$

Because target verification and draft generation proceed concurrently, the target-side forward pass overlaps with both network communication and draft-side computation.

After verification completes, the target computes the accepted length $L_n^{(i)}$ and commits the verified tokens

$$\mathbf{p}_{n+1}^{(i)} = \text{Commit}\left(\mathbf{p}_n^{(i)}, \mathbf{y}_n^{(i)}, L_n^{(i)}\right). \quad (22)$$

The target then determines whether the draft tokens remains reusable. If the verified tokens is still consistent with the speculative suffix, the target retains the unmatched suffix for the next round; otherwise, it discards the stale tokens. Formally, letting $\text{Suffix}(\mathbf{d}, \ell)$ denote the suffix of $\mathbf{d}$ after the first ℓ accepted tokens, the next-round cached tokens is

$$\mathbf{d}_{n+1}^{(i)} = \begin{cases} \text{Suffix}\left(\mathbf{d}_n^{(i)}, L_n^{(i)}\right), & \text{if } \text{Consistent}\left(\mathbf{p}_{n+1}^{(i)}, \mathbf{d}_n^{(i)}\right) = 1, \\ \emptyset, & \text{otherwise.} \end{cases} \quad (23)$$

Thus, draft tokens are treated purely as proposals; only target-verified tokens can enter the committed prefix.

Unified verification interface. The target does not introduce a separate verification algorithm for remote drafts. Instead, it converts remote speculative tokens into the tree-structured verification input used by **sglang** speculative decoding. Let $\mathcal{T}_n^{(i)}$ denote the verification tree associated with request i at round n . In the common case of a single remote tokens, $\mathcal{T}_n^{(i)}$ degenerates into a chain of length γ and verified by the same target-side procedure. Hence, SPECTRE reuses the existing verification machinery and modifies only the source of speculative proposals.

Asynchronous reply handling. Remote draft replies may be delayed, reordered, or correspond to stale decoding states. Therefore, each reply carries the identifier (i, n) . A returned tokens $\tilde{\mathbf{d}}_n^{(i)}$ is accepted only if both its request identity and round identity match the current target-side state:

$$\text{Valid}(\tilde{\mathbf{d}}_n^{(i)}) = 1 \iff (i, n) = (i_n^*, n_n^*), \quad (24)$$

where (i_n^*, n_n^*) is the active request-round pair maintained by the target. Replies failing Eq. (24) are dropped. This ensures correctness under asynchronous arrivals and fluctuating network delay.

Fallback and circuit breaker. To prevent remote instability from reducing serving throughput, the target includes two fallback mechanisms. First, if the target-side scheduler determines that speculative expansion should be disabled, the request falls back to standard one-token decoding, i.e.,

$$\mathbf{d}_n^{(i)} = \emptyset \implies \mathbf{y}_n^{(i)} = \mathbf{p}_n^{(i)} \oplus [\text{one target-decoded token}], \quad (25)$$

where $\oplus$ denotes concatenation. Second, the target maintains a circuit breaker for the remote draft service. Let m_n be an indicator of whether the draft reply for round n arrives before the timeout threshold, and let

$$c_n = \begin{cases} c_{n-1} + 1, & m_n = 0, \\ c_{n-1}, & m_n = 1. \end{cases} \quad (26)$$

If $c_n \geq C_{\max}$, the target disables remote speculation for a cooldown period of H rounds:

$$\mathbf{d}_{n+h}^{(i)} = \emptyset, \quad h = 1, \dots, H. \quad (27)$$

After the cooldown period, the target probes the remote draft service again and re-enables speculation only if timely replies are restored. This design keeps the target-side execution stable even when the remote draft service is temporarily unavailable or communication becomes unreliable.

A.2 Draft side

The draft server is implemented as a persistent speculative state machine. For each active request i , it maintains a local state

$$\mathcal{S}_n^{(i)} = (\mathbf{h}_n^{(i)}, \mathbf{d}_n^{(i)}, \mathbf{k}_n^{(i)}), \quad (28)$$

where $\mathbf{h}_n^{(i)}$ is the current decoded tokens, $\mathbf{d}_n^{(i)}$ is the speculative tokens, and $\mathbf{k}_n^{(i)}$ denotes the KV cache. This state persists across multiple synchronization rounds.

Message handling and update. At round n , the draft receives a synchronization message

$$\mathbf{m}_n^{(i)} = (i, n, \mathbf{p}_n^{(i)}), \quad (29)$$

where $\mathbf{p}_n^{(i)}$ is the verified prefix from the target. If multiple messages for the same request accumulate, only the latest one is retained.

The draft reconciles the target prefix $\mathbf{p}_n^{(i)}$ with its local decoded tokens $\mathbf{h}_n^{(i)}$ by finding the first divergence point

$$\delta_n^{(i)} = \min\{t : \mathbf{p}_n^{(i)}[t] \neq \mathbf{h}_n^{(i)}[t]\}. \quad (30)$$

State recovery. Based on $\delta_n^{(i)}$, the draft updates its state as follows:

$$\mathcal{S}_n^{(i)} \leftarrow \begin{cases} \mathcal{S}_n^{(i)}, & \delta_n^{(i)} = \gamma \text{ (no divergence)}, \\ \text{Rollback}(\mathcal{S}_n^{(i)}, \delta_n^{(i)}), & \text{otherwise} \end{cases} \quad (31)$$

where Rollback truncates the speculative suffix and frees the corresponding KV cache, while Rebuild reconstructs the state from the verified prefix.

Speculative generation. After reconciliation, the draft generates a speculative tokens of length γ :

$$\mathbf{d}_{n+1}^{(i)} = \text{Decode}_D\left(\mathbf{p}_n^{(i)}, \mathbf{k}_n^{(i)}, \gamma\right), \quad (32)$$

and returns it to the target. The request is then paused with its KV cache preserved:

$$\mathcal{S}_{n+1}^{(i)} \leftarrow \text{Pause}\left(\mathcal{S}_n^{(i)}\right). \quad (33)$$

Prompt compression. When the target-side compute significantly exceeds the draft-side capacity such that $\gamma T_D \gtrsim T_T$, the draft applies prompt compression to reduce latency. Given an input prompt of length S , the compressed prompt is

$$\mathbf{p}' = \left[\mathbf{p}_{:\lfloor \frac{\gamma}{2} S \rfloor}, \mathbf{p}_{-\lfloor \frac{\gamma}{2} S \rfloor+1:} \right]. \quad (34)$$

The draft then performs decoding conditioned on $\mathbf{p}'$ instead of the full prompt, reducing prefill and decoding cost.

Fair scheduling. To prevent starvation of regular user requests, the draft enforces a lightweight fairness constraint. Let c_n denote the number of consecutive speculative decoding steps. The scheduler updates

$$cnt_n = \begin{cases} cnt_{n-1} + 1, & \text{speculative step,} \\ 0, & \text{regular step.} \end{cases} \quad (35)$$

When $cnt_n \geq K$, the scheduler prioritizes one round of regular requests:

$$\text{if } cnt_n \geq K \Rightarrow \text{schedule regular requests and reset } cnt_n \leftarrow 0, \quad (36)$$

with $K = 10$.

Latency mismatch. Due to the variability of real-world systems, the condition

$$\gamma T_D^{\text{mix}} > T_T \quad (37)$$

may still occur. In this case, the system switches to a conservative mode where the target waits for the draft to finish generating the full speculative sequence before verification. Since draft generation becomes the bottleneck, the per-round latency is dominated by the draft side and can be approximated as

$$T_{\text{round}} \approx \gamma T_D^{\text{mix}}. \quad (38)$$

This ensures correct execution while avoiding unstable behavior under severe latency mismatch.

A.3 Communication layer

SPECTRE uses a custom asynchronous messaging backend built on ZeroMQ. The target acts as a central router, while each draft server acts as a dealer endpoint with its own identity. This routing model allows the target to direct requests to a specific draft worker and to track which draft workers are currently alive.

To support both single-node and multi-node deployments efficiently, the communication layer uses local inter-process transport on a single machine and TCP transport across machines. The implementation also separates normal data traffic from control traffic. In particular, draft workers periodically send heartbeat messages to the target, and the target removes workers from its active registry if their heartbeat expires. This provides a simple liveness mechanism without requiring an external coordination service.

The messaging backend is implemented as a multi-threaded C++ extension rather than a pure Python queue. The goal is to minimize Python overhead on the critical path. Sending, receiving, unpacking, and endpoint monitoring are performed by separate background threads, while Python interacts only with already unpacked

request batches. Messages are serialized in batches using a compact binary format, which substantially reduces framing overhead compared with per-request transmission.

The transport layer also includes explicit defenses against pathological runtime conditions. It uses bounded queues to avoid unbounded memory growth under backpressure, discards stale buffered messages that have exceeded a timeout, and records malformed payloads and shutdown-time drops for debugging. These details are not algorithmically central, but they are important for making the system reliable in long-running serving workloads where network delay, process restarts, and temporary overload are unavoidable.

Overall, SPECTRE is implemented as a pipelined remote speculative system in which the target server owns correctness and verification, the draft server maintains persistent speculative state with efficient rollback, and the communication layer provides asynchronous, version-aware, and fault-tolerant transport. The practical efficiency of the system comes from overlapping communication with target-side computation, reusing paused draft states across rounds, and restricting expensive cache rebuilds to the cases where the target and draft histories truly diverge.

B Related Work

B.1 Speculative Decoding

Speculative decoding accelerates auto-regressive generation by letting a lightweight draft model propose multiple candidate tokens and using a larger target model to verify them with fewer target-side decoding steps. Canonical methods preserve the target distribution through rejection sampling, and subsequent work improves draft quality, verification efficiency, and proposal policies within this sequential draft-then-verify framework [17, 22–30].

More recent work increases concurrency between drafting and verification instead of treating them as strictly sequential stages. PEARL reduces the bubbles between successive verification steps through pre-verify and post-verify scheduling, thereby alleviating the mutual waiting between drafting and verification [6]. SSD pushes this further by predicting likely verification outcomes and pre-speculating for multiple such outcomes during an ongoing verification [31]. MineDraft studies batch-parallel speculative execution to hide drafting latency under verification and improve overlap at the system level [19]. Our method is most closely related to recent work on parallel speculative decoding, and introduces a hybrid ordinary-parallel design for batched target-side execution.

B.2 LLM Serving and Resource Utilization

A growing body of systems work studies LLM serving through better scheduling, disaggregation, and resource orchestration to improve GPU utilization under heterogeneous and time-varying workloads [4, 32–38]. These works are motivated by the observation that serving load is often uneven across requests, execution phases, and co-deployed models, leaving some resources underutilized under static provisioning. Among them, Weaver is closest to our motivation: it identifies hot/cold demand skew in multi-LLM serving and exploits idle capacity from cold models to assist hot models through attention offloading [39, 40]. Our work is similarly motivated by skewed utilization in multi-model serving, but uses lightly loaded small-model services to provide speculative drafts for heavily loaded large-model services.

C Detailed Throughput Analysis

This appendix provides the detailed derivation of the throughput models used in Section 2. We consider the decode stage with batch size B . Let T_D and T_T denote the latency of one draft-model forward step and one target-model verification pass, respectively. Let L denote the accepted length, and let γ denote the number of speculative tokens processed in one target verification. We focus on the regime

$$\gamma T_D < T_T, \tag{39}$$

where draft-side generation can potentially be overlapped with target-side verification.

Ordinary speculative decoding. As shown in the top pipeline of Fig. 2, ordinary speculative decoding executes draft generation and target verification sequentially across rounds. After initialization, each round verifies a speculative segment of length γ and produces one target-side bonus token. Since this bonus token becomes the first token of the next speculative segment, the draft model only needs to generate $\gamma - 1$ new speculative tokens in each subsequent round. Therefore, the per-round latency is

$$T_{\text{ord}} = T_T + (\gamma - 1)T_D. \quad (40)$$

Since each round produces BL accepted tokens for a batch of size B , the ordinary speculative decoding throughput is

$$\text{Thr}_{\text{ord}} = \frac{BL}{T_{\text{ord}}} = \frac{BL}{T_T + (\gamma - 1)T_D}. \quad (41)$$

Parallel speculative decoding. As shown in the bottom pipeline of Fig. 2, parallel speculative decoding overlaps target verification and draft generation across different rounds. At round n , the target verifies the speculative segment prepared in round $n - 1$, while the draft simultaneously prepares the speculative segment for round $n + 1$. Unlike ordinary speculative decoding, the draft model must generate γ new speculative tokens in each round, rather than $\gamma - 1$, because the first token of the next segment is generated before the current target verification result is available.

Let r be the fraction of requests whose pre-generated speculative continuation becomes invalid after verification. These requests cannot reuse the prepared continuation and fall back to autoregressive decoding, contributing Br tokens per batch. The remaining fraction $1 - r$ successfully reuses the prepared speculative segment and contributes $B(1 - r)L$ tokens per batch. Thus, the total number of output tokens per round is

$$B[r + (1 - r)L]. \quad (42)$$

Under Eq. (39), draft-side generation is hidden by target-side verification, so the per-round latency is approximated by

$$T_{\text{par}} \approx T_T. \quad (43)$$

The throughput of parallel speculative decoding is therefore

$$\text{Thr}_{\text{par}} = \frac{B[r + (1 - r)L]}{T_{\text{par}}} \approx \frac{B[r + (1 - r)L]}{T_T}. \quad (44)$$

Critical fallback ratio. We derive the condition under which ordinary speculative decoding outperforms parallel speculative decoding:

$$\text{Thr}_{\text{ord}} > \text{Thr}_{\text{par}}. \quad (45)$$

Substituting Eq. (41) and Eq. (44), and cancelling B , gives

$$\frac{L}{T_T + (\gamma - 1)T_D} > \frac{r + (1 - r)L}{T_T}. \quad (46)$$

Since $r + (1 - r)L = L - r(L - 1)$, we have

$$LT_T > (L - r(L - 1))(T_T + (\gamma - 1)T_D) \quad (47)$$

$$= LT_T + L(\gamma - 1)T_D - r(L - 1)(T_T + (\gamma - 1)T_D). \quad (48)$$

Rearranging yields

$$r(L - 1)(T_T + (\gamma - 1)T_D) > L(\gamma - 1)T_D. \quad (49)$$

Assuming $L > 1$, ordinary speculative decoding is preferable when

$$r > \frac{(\gamma - 1)LT_D}{(T_T + (\gamma - 1)T_D)(L - 1)} = r^*. \quad (50)$$

Equivalently, parallel speculative decoding is beneficial only when $r \leq r^*$. This threshold captures the core trade-off in Fig. 2: parallel execution hides draft latency, but its benefit disappears when too many pre-generated continuations become invalid. SPECTRE therefore uses this condition to guide its hybrid ordinary-parallel execution strategy.

Model (bs)	Method	GSM8K		Math500		Minerva Math		ShareGPT		LongBench	
		Tok/s	Speedup	Tok/s	Speedup	Tok/s	Speedup	Tok/s	Speedup	Tok/s	Speedup
Qwen3-32B (bs=1)	AR	51.20	1.00×	51.17	1.00×	51.15	1.00×	47.12	1.00×	47.64	1.00×
	EAGLE3	86.54	1.69×	84.07	1.64×	83.26	1.63×	72.17	1.53×	75.37	1.58×
	PEARL	92.13	1.80×	90.59	1.77×	96.26	1.88×	109.67	2.33×	89.48	1.88×
	MineDraft	49.53	0.97×	50.21	0.98×	49.71	0.97×	—	—	—	—
	Standalone	117.10	2.29×	115.03	2.25×	118.71	2.32×	112.92	2.40×	106.65	2.24×
	SPECTRE	120.81	2.36×	120.25	2.35×	121.07	2.37×	146.96	3.12×	109.33	2.29×
	Δ (%)		$\uparrow$ 3.1%		$\uparrow$ 4.4%		$\uparrow$ 2.2%		$\uparrow$ 30.0%		$\uparrow$ 2.2%
Qwen3-32B (bs=16)	AR	632.18	1.00×	629.20	1.00×	627.41	1.00×	548.85	1.00×	347.39	1.00×
	EAGLE3	966.58	1.53×	968.45	1.54×	891.35	1.42×	693.24	1.26×	280.41	0.81×
	PEARL	897.05	1.42×	919.35	1.46×	859.95	1.37×	843.96	1.54×	515.64	1.48×
	MineDraft	456.31	0.72×	479.99	0.76×	450.68	0.72×	—	—	—	—
	Standalone	1122.98	1.78×	1094.45	1.74×	1118.89	1.78×	933.13	1.70×	558.48	1.61×
	SPECTRE	1261.15	1.99×	1294.87	2.06×	1172.97	1.87×	1133.64	2.07×	605.73	1.74×
	Δ (%)		$\uparrow$ 11.8%		$\uparrow$ 18.4%		$\uparrow$ 5.1%		$\uparrow$ 21.8%		$\uparrow$ 8.1%
DS-32B (bs=1)	AR	52.37	1.00×	52.56	1.00×	52.55	1.00×	49.63	1.00×	47.78	1.00×
	PEARL	107.03	2.04×	108.42	2.06×	108.82	2.07×	113.76	2.29×	61.26	1.28×
	MineDraft	60.97	1.16×	64.46	1.23×	64.39	1.23×	—	—	—	—
	Standalone	119.00	2.27×	123.99	2.36×	124.49	2.37×	121.66	2.45×	119.64	2.50×
	SPECTRE	139.62	2.67×	142.48	2.71×	141.52	2.69×	147.60	2.97×	131.99	2.76×
	Δ (%)		$\uparrow$ 17.6%		$\uparrow$ 14.8%		$\uparrow$ 13.5%		$\uparrow$ 21.2%		$\uparrow$ 10.4%
DS-32B (bs=16)	AR	623.74	1.00×	623.46	1.00×	621.02	1.00×	588.30	1.00×	389.52	1.00×
	PEARL	992.22	1.59×	1002.69	1.61×	930.88	1.50×	617.22	1.05×	186.68	0.48×
	MineDraft	563.63	0.90×	591.26	0.95×	581.86	0.94×	—	—	—	—
	Standalone	1362.28	2.18×	1420.58	2.28×	1371.41	2.21×	888.04	1.51×	558.48	1.43×
	SPECTRE	1469.75	2.36×	1539.59	2.47×	1383.45	2.23×	1111.64	1.89×	805.51	2.07×
	Δ (%)		$\uparrow$ 8.3%		$\uparrow$ 8.3%		$\uparrow$ 0.9%		$\uparrow$ 25.2%		$\uparrow$ 44.8%

Table 4 Results for small batch sizes (bs=1 and bs=16). We report throughput (Tok/s) and speedup over AR. SPECTRE consistently achieves the best performance across all settings.

D Additional Experiment

D.1 Experiment for small batch size

We report results for small batch sizes (bs=1 and 16) in Table 4. At bs=1, SPECTRE consistently achieves the best performance across all datasets and both model families. On Qwen3-32B, speedups ranging from 1.74× to 3.12× and outperform the SOTA by up to 30.0% on ShareGPT at bs=1, On DeepSeek-R1-Distill-Qwen-32B, speedups ranging from 1.89× to 2.97× and outperform the SOTA by up to 44.8% on LongBench at bs=16. This indicates that even in the latency-sensitive regime where speculative parallelism is limited, SPECTRE can still effectively utilize draft-side computation. We also observe that some baselines degrade significantly in this regime. For example, EAGLE3 drops below 1.0× on LongBench at bs=16, and MineDraft consistently underperforms due to system inefficiencies. In contrast, SPECTRE exhibits stable gains across both reasoning and long-context workloads. Overall, these results demonstrate that SPECTRE not only scales well to large batch sizes (as shown in table. 1), but also delivers robust and consistent improvements in small-batch settings.

D.2 Throughput at Temperature 0.6

Table 5 reports throughput results under sampling temperature 0.6, which we adopt to follow the default generation configurations, ensuring a realistic evaluation setting. Overall, SPECTRE consistently achieves strong performance, with particularly notable gains over SOTA on ShareGPT and LongBench (up to 50.9% and 41.9% when target model is DeepSeek-R1-Distill-Qwen-32B, bs=128), demonstrating its outstanding performance. As batch size increases, the advantage of SPECTRE becomes more pronounced: for Qwen3-32B, improvements grow from marginal gains at bs = 1 (e.g., +0.5% on GSM8K) to substantial gains at larger batch sizes (e.g., +20.7% at bs = 64), while for DeepSeek-R1-Distill-Qwen-32B, gains exceed 34.1% on LongBench at bs $\geq$ 32, indicating superior scalability in high-throughput serving regimes. These results collectively validate that SPECTRE maintains consistent throughput improvements and favorable scaling behavior under realistic sampling conditions.

E Experiment Settings

E.1 Detailed Experimental Setup for Table.1,4,3

Workload generation. To emulate realistic online serving traffic, we generate request arrivals according to a Poisson process. Specifically, the inter-arrival time between two consecutive requests is sampled as

$$\Delta t \sim \text{Exp}(1/\text{QPS}), \quad (51)$$

where QPS denotes the target request rate. This setup approximates bursty and stochastic production workloads.

Batch size control. We control the effective batch size through the maximum request concurrency parameter (`max-concurrency`). To efficiently obtain stable measurements across different regimes, we jointly adjust QPS and the number of evaluation samples for each batch size. Concretely, we use

- Batch sizes: $\{1, 16, 32, 64, 128\}$,
- QPS: $\{1, 4, 8, 16, 32\}$,
- Number of samples: $\{8, 64, 128, 256, 512\}$,

respectively. Larger batch sizes correspond to higher request rates and more samples to ensure sufficient system saturation and statistical stability.

Decoding configuration. We implement SPECTRE in `SGLang v0.5.7`. Speculative decoding is a lossless acceleration technique; therefore, we focus on throughput rather than output quality. We fix the output length of each request to 1024 tokens. To avoid early termination due to end-of-sequence tokens, we enable `ignore_eos` during decoding. This ensures that all methods are evaluated under identical decoding lengths and eliminates variance caused by early stopping.

Dataset-specific setup. For mathematical reasoning datasets (GSM8K, Math500, and Minerva Math), we construct few-shot prompts by prepending the solutions of the first five problems as demonstrations. This provides consistent reasoning context across all methods. For ShareGPT, we follow the official `sglang` benchmarking pipeline and use `bench_serving.py` to generate realistic conversational workloads. The input prompt length is fixed to 4000 tokens to simulate long-context scenarios.

Model parallelism. In these experiments, both the target and draft models are deployed with TP size 1. As a result, there is no significant compute imbalance between the two models. Consequently, we do not apply prompt compression in this setting, allowing us to isolate the effect of the speculative decoding strategy itself.

E.2 Detailed experimental setup for Qwen3-235B-A22B

We evaluate SPECTRE on a large-model setting using the Qwen3-235B-A22B / Qwen3-0.6B pair, as shown in Fig. 4. In this configuration, the target model is deployed with TP size 8, while the draft model uses TP=1, reflecting the realistic compute imbalance between large and small models in production systems.

To emulate online serving conditions, we set the request rate to $\text{QPS} = 32$ and the maximum request concurrency to 128. Each request generates 1024 output tokens, and `ignore_eos` is enabled to ensure consistent decoding length across all methods.

Due to the substantial compute gap between the target and draft models, we apply draft-side context compression to reduce latency. Specifically, given a prompt of length S , we retain only a fraction p of tokens by keeping the first $pS/2$ tokens and the last $pS/2$ tokens, with $p = 0.1$ in our experiments. This reduces the draft-side prefill and decoding cost, enabling better overlap between draft generation and target-side verification.

E.3 Mixed Draft Traffic Setup

We evaluate system robustness under mixed draft-side traffic by fixing the target workload and varying the background load on the draft server. The target request rate is fixed, with a maximum request concurrency of 128. On the draft side, we inject background user traffic with request rates of 1, 2, 4, and 8 QPS. For each draft QPS setting, a total of 64 requests are issued, while the maximum request concurrency of the draft server is fixed at 256. For each background load level, we first measure the draft throughput, and then evaluate the resulting target throughput under the same conditions.

E.4 System Benefit Evaluation Setup

We evaluate system-level benefit efficiency in terms of dollar/1000s/GPU, which measures the revenue generated per unit GPU time by jointly accounting for both target and draft serving. The target workload is fixed to 32 QPS with a maximum request concurrency of 128. For SPECTRE and PEARL, the draft model is deployed on a separate GPU (TP1), while for other baselines (e.g., EAGLE3 and Standalone), the draft and target models are co-located on the same GPU. All models use TP1 unless otherwise specified, except Qwen3-235B-A22B, which runs with TP8. To compute the system-level benefit, we explicitly account for both throughput and resource cost. For Qwen3-32B and DeepSeek-R1-Distill-Qwen-32B, the target model is deployed with TP1 and paired with a smaller draft model (Qwen3-0.6B or DeepSeek-R1-Distill-Qwen-1.5B) on a separate GPU (also TP1), resulting in a total cost of two GPUs. For Qwen3-235B-A22B, the target model is deployed across 8 GPUs (TP8), together with a draft model on an additional GPU (TP1), leading to a total cost of nine GPUs. For AR, EAGLE3, Standalone, and SPECTRE, we define the benefit as the total served tokens multiplied by token price, i.e., (target throughput + draft throughput) $\times$ price, normalized by the number of GPUs. In contrast, for PEARL, since the draft model does not serve regular user traffic, its contribution to revenue is excluded, and the benefit is computed solely based on target throughput divided by the total GPU count. For the Qwen3-235B-A22B (TP=8) / Qwen3-0.6B (TP=1) configuration, the target-side compute is substantially stronger, making the draft side the bottleneck; accordingly, we set a lower background load on the draft server (4 QPS with maximum concurrency 100). For other settings, we use a higher background draft load (8 QPS with maximum concurrency 256). Token pricing follows public API rates³ (as of 04/23/2026): Qwen3-32B (\$3/M), Qwen3-0.6B (\$0.45/M), Qwen3-235B-A22B (\$3/M), DeepSeek-R1-Distill-Qwen-32B (\$0.9/M), and DeepSeek-R1-Distill-Qwen-1.5B (free).

³<https://help.aliyun.com/zh/model-studio/model-pricing>

Model (bs)	Method	GSM8K		Math500		Minerva Math		ShareGPT		LongBench	
		Tok/s	AR ×	Tok/s	AR ×	Tok/s	AR ×	Tok/s	AR ×	Tok/s	AR ×
Qwen3-32B (bs=1)	AR	51.39	1.00×	51.58	1.00×	51.57	1.00×	50.96	1.00×	49.12	1.00×
	EAGLE3	81.47	1.59×	88.01	1.71×	79.20	1.54×	100.99	1.98×	74.58	1.52×
	PEARL	84.55	1.65×	95.36	1.85×	91.66	1.78×	85.77	1.68×	81.65	1.66×
	Standalone	113.29	2.20×	114.72	2.22×	106.11	2.06×	109.61	2.15×	96.13	1.96×
	SPECTRE	113.80	2.21×	114.89	2.23×	120.50	2.34×	137.75	2.70×	96.32	1.96×
	Δ		↑ 0.5%		↑ 0.4%		↑ 13.6%		↑ 25.6%		↑ 0.0%
Qwen3-32B (bs=16)	AR	674.52	1.00×	673.22	1.00×	673.21	1.00×	556.52	1.00×	429.98	1.00×
	EAGLE3	1007.62	1.49×	1000.59	1.49×	901.82	1.34×	<u>816.34</u>	<u>1.47×</u>	<u>527.65</u>	<u>1.23×</u>
	PEARL	808.18	1.20×	845.22	1.26×	769.13	1.14×	607.71	1.09×	490.99	1.14×
	Standalone	1064.48	1.58×	1104.52	1.64×	1058.97	1.57×	756.03	1.36×	504.17	1.17×
	SPECTRE	1168.70	1.73×	1232.49	1.83×	1158.37	1.72×	829.70	1.49×	624.43	1.45×
	Δ		↑ 9.5%		↑ 11.6%		↑ 9.5%		↑ 1.4%		↑ 17.8%
Qwen3-32B (bs=32)	AR	1132.42	1.00×	1125.59	1.00×	1121.91	1.00×	836.60	1.00×	583.38	1.00×
	EAGLE3	1456.27	1.29×	1521.70	1.35×	1468.74	1.31×	<u>1064.58</u>	<u>1.27×</u>	<u>675.56</u>	<u>1.16×</u>
	PEARL	1169.81	1.03×	1147.89	1.02×	1136.12	1.01×	749.78	0.90×	521.78	0.89×
	Standalone	1528.09	1.35×	1562.91	1.39×	1542.32	1.37×	752.51	0.90×	498.76	0.85×
	SPECTRE	1732.45	1.53×	1838.71	1.63×	1745.68	1.56×	1119.59	1.34×	718.28	1.23×
	Δ		↑ 13.3%		↑ 17.3%		↑ 13.9%		↑ 5.5%		↑ 6.0%
Qwen3-32B (bs=64)	AR	1692.39	1.00×	1690.82	1.00×	1679.18	1.00×	904.59	1.00×	608.44	1.00×
	EAGLE3	<u>1962.39</u>	<u>1.16×</u>	<u>1343.24</u>	<u>0.79×</u>	<u>1866.97</u>	<u>1.11×</u>	<u>1212.35</u>	<u>1.34×</u>	<u>697.02</u>	<u>1.15×</u>
	PEARL	991.17	0.59×	1386.43	0.82×	1341.77	0.80×	835.76	0.92×	480.88	0.79×
	Standalone	1846.31	1.09×	1852.00	1.10×	1798.87	1.07×	778.18	0.86×	529.35	0.87×
	SPECTRE	2372.58	1.40×	2336.98	1.38×	2206.57	1.31×	1237.14	1.37×	759.21	1.25×
	Δ		↑ 20.7%		↑ 25.5%		↑ 18.0%		↑ 2.2%		↑ 8.7%
Qwen3-32B (bs=128)	AR	2190.45	1.00×	2194.08	1.00×	1923.54	1.00×	878.22	1.00×	603.63	1.00×
	EAGLE3	<u>2192.51</u>	<u>1.00×</u>	<u>2204.53</u>	<u>1.00×</u>	<u>1993.04</u>	<u>1.04×</u>	<u>1273.50</u>	<u>1.45×</u>	528.53	0.88×
	PEARL	637.69	0.29×	887.87	0.40×	817.81	0.43×	933.93	1.06×	472.93	0.78×
	Standalone	1930.04	0.88×	1962.45	0.89×	1860.39	0.97×	838.17	0.95×	702.76	1.16×
	SPECTRE	2608.87	1.19×	2659.45	1.21×	2426.95	1.26×	1313.67	1.50×	754.00	1.25×
	Δ		↑ 19.0%		↑ 21.0%		↑ 21.2%		↑ 3.4%		↑ 7.8%
DS-32B (bs=1)	AR	50.72	1.00×	50.90	1.00×	50.83	1.00×	50.96	1.00×	48.82	1.00×
	PEARL	90.00	1.77×	86.22	1.69×	94.97	1.87×	86.32	1.69×	99.72	2.04×
	Standalone	118.55	2.34×	114.83	2.26×	115.44	2.27×	123.26	2.42×	115.58	2.37×
	SPECTRE	126.97	2.50×	120.07	2.36×	120.48	2.37×	140.60	2.76×	147.85	3.03×
	Δ		↑ 6.8%		↑ 4.4%		↑ 4.4%		↑ 14.1%		↑ 27.8%
DS-32B (bs=16)	AR	682.53	1.00×	681.51	1.00×	681.78	1.00×	547.01	1.00×	428.59	1.00×
	PEARL	936.72	1.37×	933.64	1.37×	901.64	1.32×	633.15	1.16×	608.60	1.42×
	Standalone	1292.60	1.89×	1355.10	1.99×	1261.70	1.85×	915.66	1.67×	637.10	1.49×
	SPECTRE	1324.66	1.94×	1460.37	2.14×	1351.04	1.98×	978.20	1.79×	787.07	1.84×
	Δ		↑ 2.6%		↑ 7.5%		↑ 7.0%		↑ 7.2%		↑ 23.5%
DS-32B (bs=32)	AR	1156.82	1.00×	1149.42	1.00×	1143.46	1.00×	853.33	1.00×	565.17	1.00×
	PEARL	1023.68	0.88×	1150.46	1.00×	1101.21	0.96×	848.33	0.99×	<u>685.79</u>	<u>1.21×</u>
	Standalone	2081.00	1.80×	1973.52	1.72×	2049.75	1.79×	933.02	1.09×	622.29	1.10×
	SPECTRE	2148.10	1.86×	2215.11	1.93×	2101.60	1.84×	1397.53	1.64×	920.98	1.63×
	Δ		↑ 3.3%		↑ 12.2%		↑ 2.5%		↑ 50.5%		↑ 34.1%
DS-32B (bs=64)	AR	1697.20	1.00×	1693.34	1.00×	1681.12	1.00×	924.19	1.00×	587.54	1.00×
	PEARL	969.03	0.57×	908.49	0.54×	1023.42	0.61×	946.97	1.02×	<u>719.87</u>	<u>1.23×</u>
	Standalone	2637.40	1.55×	2697.26	1.59×	2494.65	1.48×	968.29	1.05×	657.09	1.12×
	SPECTRE	2819.25	1.66×	2875.32	1.70×	2708.55	1.61×	1446.21	1.56×	966.69	1.65×
	Δ		↑ 7.1%		↑ 6.9%		↑ 8.8%		↑ 48.6%		↑ 34.1%
DS-32B (bs=128)	AR	2195.28	1.00×	2200.47	1.00×	2198.42	1.00×	903.00	1.00×	582.58	1.00×
	PEARL	637.58	0.29×	637.59	0.29×	768.77	0.35×	987.80	1.09×	<u>683.28</u>	<u>1.17×</u>
	Standalone	2734.99	1.25×	2818.41	1.28×	2595.55	1.18×	996.86	1.10×	655.23	1.12×
	SPECTRE	3199.90	1.46×	3249.42	1.48×	2969.55	1.35×	1497.50	1.66×	967.50	1.66×
	Δ		↑ 16.8%		↑ 15.6%		↑ 14.4%		↑ 50.9%		↑ 41.9%

Table 5 Throughput results at temperature 0.6 for Qwen3-32B and DeepSeek-R1-Distill-Qwen-32B across all batch sizes. For each benchmark, the SOTA result is underlined. Δ reports the relative improvement of SPECTRE over the SOTA method.